Proiectarea Algoritmilor

Curs 8 – Drumuri de cost minim de sursă unică



Bibliografie

[1] Giumale – Introducere în Analiza Algoritmilor cap. 5.4, 5.4.1

[2] Cormen – Introducere în Algoritmi cap. Heap-uri binomiale (20), Heap-uri Fibonacci (21), Drumuri minime de sursă unica - primele 2 subcapitole (25.1 și 25.2)

[3] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/06PriorityQueues.pdf

[4] Heap Fibonacci: https://www.cs.usfca.edu/~galles/JavascriptVisual/FibonacciHeap.html

[5] Fibonacci heap: http://www.growingwiththeweb.com/data-structures/fibonacci-heap/overview/

Obiective

- “Descoperirea” algoritmilor de identificare a drumurilor de cost minim.
- Recunoașterea caracteristicilor acestor algoritmi.
- Identificarea structurilor de date necesare pentru reducerea complexității acestor algoritmi.



Drumuri de cost minim

- G = (V,E) un graf, iar w:E → $\Re$ o funcție de cost asociată arcelor grafului (w(u,v) = costul arcului (u,v)).
- Cost(u..v) = costul drumului u..v (este aditiv – costul drumului = suma costurilor arcelor).
- Variante:
  1. Drumuri punct – multipunct: pentru un nod dat s ∈ V, să se găsească un drum de cost minim de la s la ∀u ∈ V; Dijkstra, Bellman-Ford
  2. Drumuri multipunct – punct: pentru un nod dat e ∈ V, să se găsească un drum de cost minim de la ∀u ∈ V la e; G^T și apoi 1
  3. Drumuri punct – punct: pentru două noduri date u și v ∈ V, să se găsească un drum u..v de cost minim; Folosind 1
  4. Drumuri multipunct – multipunct: ∀u, v ∈ V, să se găsească un drum u..v de cost minim. Floyd-Warshall
  5. Drumuri de cost maxim!

Temă de gândire pentru acasă – posibil subiect de examen!

Optimalitatea drumurilor minime

- Lemă 25.1 (Subdrumurile unui drum minim sunt drumuri optimale): G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. Fie p = $v_1 v_2 ... v_k$ un drum optim de la v_1 la v_k. Atunci pentru orice i și j cu $1 \leq i \leq j \leq k$, subdrumul lui p de la v_i la v_j este un drum minim.

- Corolar 25.2: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Atunci costul optim al acestui drum poate fi scris ca $\delta(s,v) = \delta(s,u) + w(u,v)$.

- Lemă 25.3: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. $\forall (u,v) \in E$ avem $\delta(s,v) \leq \delta(s,u) + w(u,v)$.

Drumuri minime de sursă unică

- Sunt concepuți pentru grafuri orientate.
- Bazați pe algoritmi de tip Greedy (Dijkstra) sau de programare dinamică (Bellman–Ford).
- Se pornește de la nodul de start și pe baza unui optim local, drumurile sunt extinse și optimizate până la soluția finală.

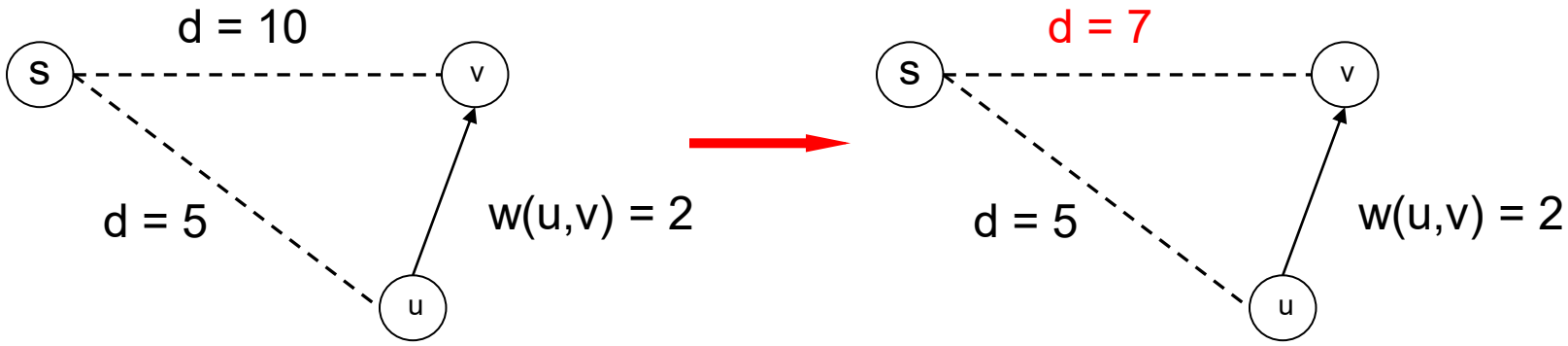


Drumuri de sursă unică - Notații

- G = (V,E);
- $s \in V$ – nodul sursă;
- $w : E \rightarrow \mathfrak{R}$ funcție de cost asociată arcelor grafului;
- cost(u..v) = costul drumului u..v (aditiv);
- d[v] = costul drumului descoperit s..v;
- δ(u,v) = costul drumului optim u..v;
  - $\delta(u,v) = \infty$ dacă $v \notin R(u)$
  - $\delta(u,v) = \Sigma w(x,y)$, $(x,y) \in$ u..v (u..v fiind drumul optim);
- p[v] = predecesorul lui v pe drumul s..v.

Relaxarea arcelor

- Relaxarea arcelor:
  - **Dacă** d[v] > d[u] + w(u,v), **atunci**
    - d[v] = d[u] + w(u,v);
    - p[v] = u



Algoritmul lui Dijkstra (I)

- Folosește o coadă de priorități în care se adaugă nodurile în funcție de distanța cunoscută în momentul respectiv de la s până la nod.
- Se folosește **NUMAI** pentru costuri pozitive ($w(u,v) > 0, \forall u,v \in V$).
- Dijkstra_generic (G,s)
  - V = nodurile lui G
  - **Cât timp** (V != ∅)
    - u = nod din V cu d[u] min
    - V = V - {u}
    - **Pentru fiecare** (v ∈ succesorii lui u) relaxare_arc(u,v)

    // optimizare drum s..v pentru v ∈ succesorilor lui u

Corectitudine Dijkstra

- Lemă 25.5: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. ∀ v ∈ V, d[v] obținut de algoritmul lui Dijkstra respectă d[v] ≥ δ(s,v). În plus, odată atinsă valoarea δ(s,v), ea nu se mai modifică.

- Lemă 25.7: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Dacă d[u] = δ(s,u) la un moment dat, atunci începând cu momentul imediat următor relaxării arcului (u,v) avem d[v] = δ(s,v).



Algoritmul lui Dijkstra (II)

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V) // inițializări
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele

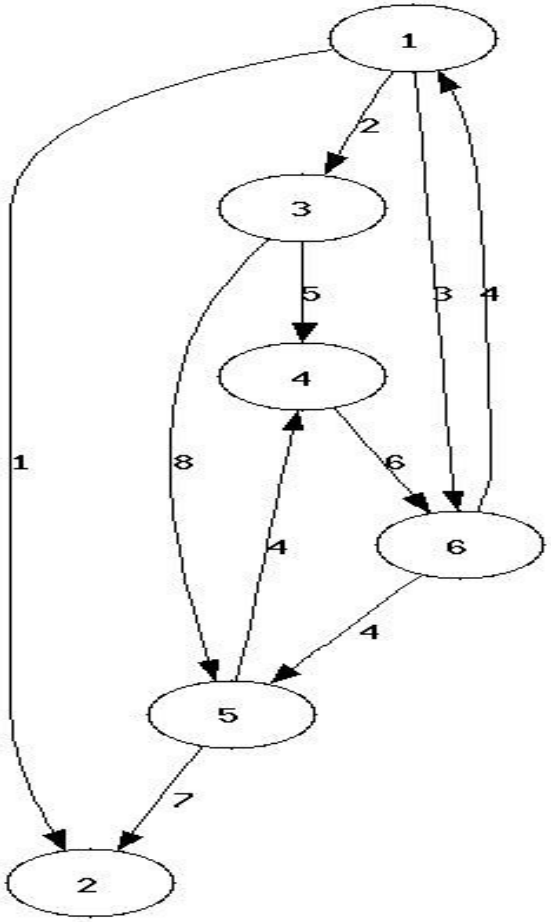
Corectitudine Dijkstra

- Teoremă. G = (V,E), w : E → $\Re$ funcție de cost asociată nenegativă. La terminarea aplicării algoritmului Dijkstra pe acest graf plecând din sursa s vom avea d[v] = δ(s,v) pentru ∀ v ∈ V.

- Dem: prin reducere la absurd se demonstrează că la scoaterea din Q a fiecărui nod v avem d[v] = δ(s,v) și egalitatea se menține și ulterior.
  - Pp. u e primul nod pt. care d[u] ≠ δ(s,u) la scoaterea din Q. u ≠ s pt. că altfel d[u] = δ(s,u) = 0 și u ∈ R(s) pt.că altfel d[u] = δ(s,u) = ∞. => La scoaterea lui u din Q, ∃ drum s..u și fie p drumul optim s..u a.î. p = s..xy..u, unde x ∉ Q iar y ∈ Q.
  - (Dacă nu impunem y ∈ Q, atunci toate nodurile până la u au fost scoase din Q și u a rămas în vârful cozii. Cum u e primul nod pentru care d[u] ≠ δ(s,u), iar calea e optimă, avem situația din Lema 25.7 => La scoaterea lui u din Q, d[u] = δ(s,u).)
  - Cum u e primul nod pt. care d[u] ≠ δ(s,u) => d[x] = δ(s,x) la momentul extragerii lui u din Q → d[y] = δ(s,y) prin relaxarea (x,y) (conf. Lema 25.7).
  - y precede u pe drumul p => d[y] = δ(s,y) ≤ δ(s,u) ≤ d[u] (conf. Lema 25.5).
  - Cum y ∈ Q la momentul scoaterii lui u din Q => d[u] ≤ d[y]
  - => d[y] = δ(s,y) = δ(s,u) = d[u] – Contrazice ipoteza! => d[u] = δ(s,u) și conf. Lema 25.5, egalitatea se menține și ulterior.



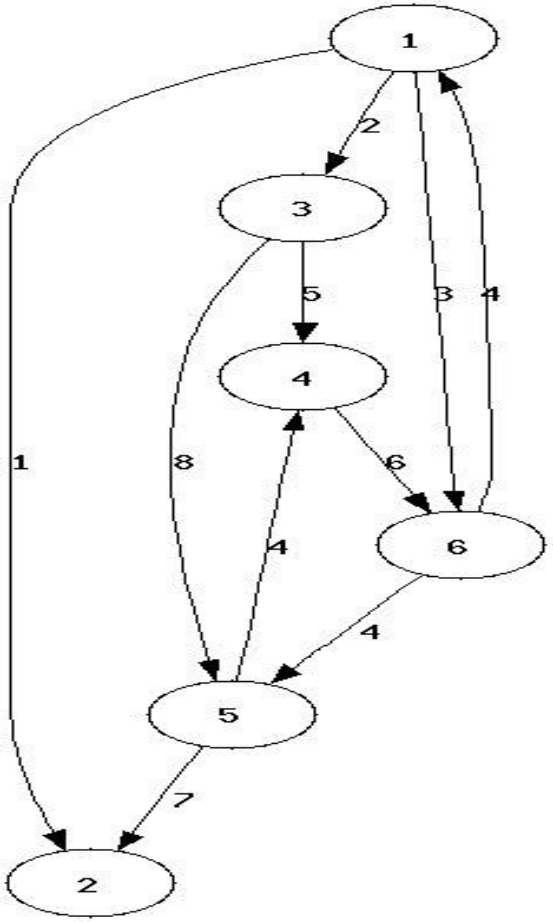
Exemplu (I)



Exemplu (II)

- d[1] = 0;
- (1): d[2] = 1; d[3] = 2; d[6] = 3;
- (2): d[4] = 7; d[5] = 10;
- (3): d[5] = 7;

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele



Complexitate Dijkstra

- Depinde de ExtrageMin – coadă cu priorități.
- Operații ce trebuie realizate pe coadă + frecvența lor:
  - insert – V;
  - delete – V;
  - conține? – E;
  - micșorează_val – E;
  - este_vidă? – V.

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută in cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q si v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // si părintele

Implementare cu vectori

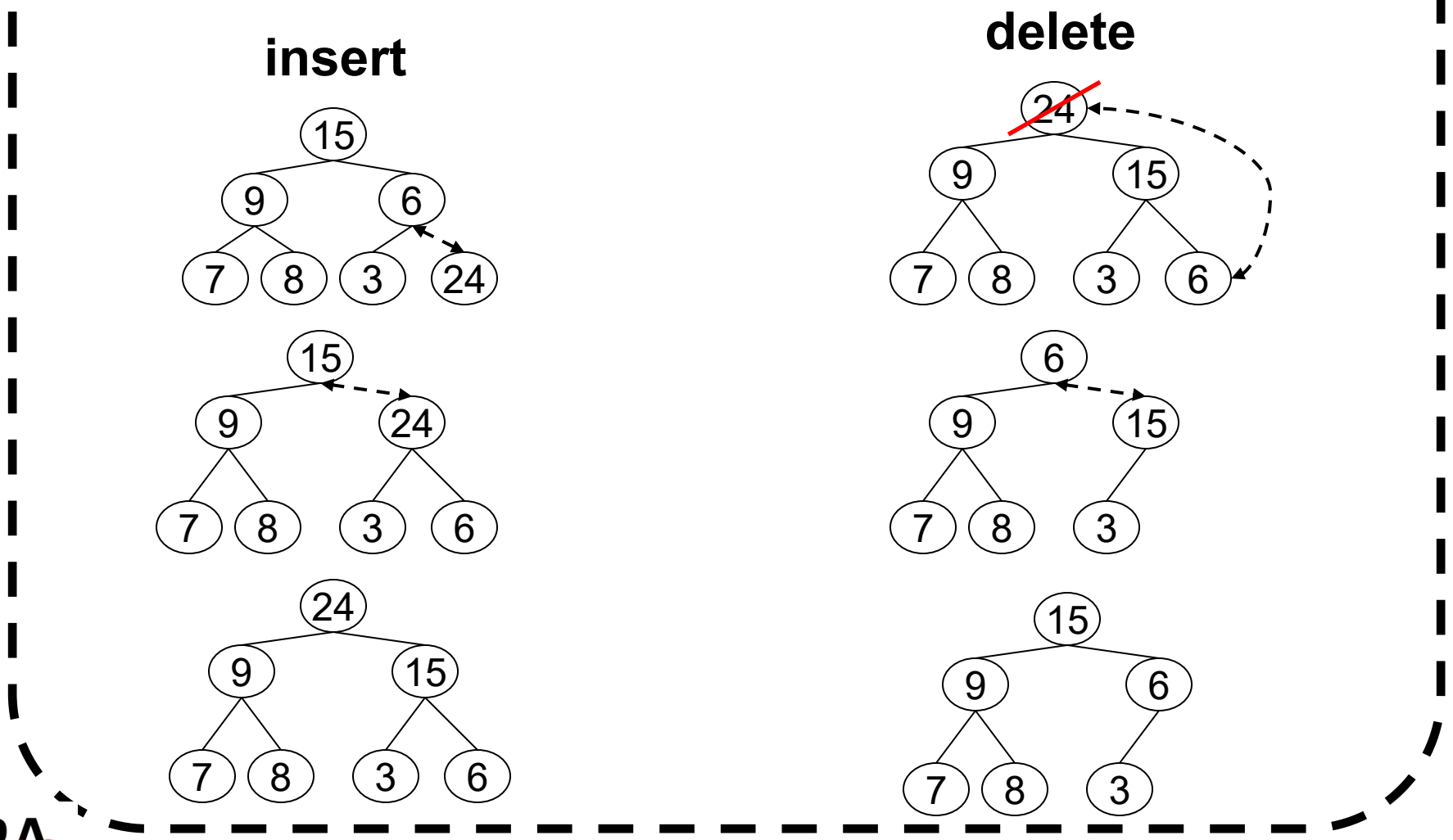
- Costuri:
  - insert – 1 * V = V;
  - delete – V * V = V^2 (necesită căutarea minimului);
  - conține? – 1 * E = E;
  - micșorează_val – 1 * E = E;
  - este_vidă? – 1 * V = V;

- Cea mai bună metodă pentru grafuri "dese" ($E \approx V^2$)!

Implementare cu heap binar

- Heap binar – structură de date de tip arbore binar + 2 constrângeri:
  - Fiecare nivel este complet; ultimul se umple de la stânga la dreapta;
  - $\forall u \in \text{Heap}$; $u \geq$ răd(st(u)) și $u \geq$ răd(dr(u)) (u este $\geq$ decât ambii copii ai săi) unde $\geq$ este o relație de ordine pe mulțimea pe care sunt definite elementele heapului.

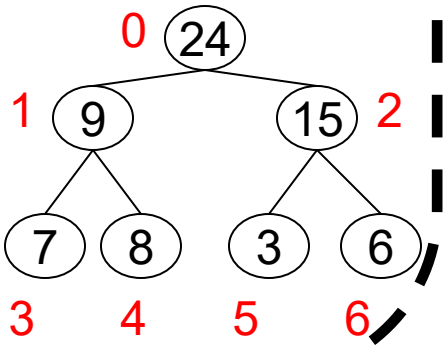
Operatii pe Heap Binar



Implementare Heap Binar

- Implementare folosind vectori.
- Poziție[i] = unde se găsește în indexul de valori elementul de pe poziția i din heap.
- Reverse[i] = unde se găsește în heap elementul de pe poziția i din valoare.
- Implementare disponibila la [3].

Index	0	1	2	3	4	5	6
Valoare	7	6	15	8	24	9	3
Poziție							
Reverse							



Heap Binar

- Costuri:
  - insert – logV * V = VlogV;
  - delete – logV * V = VlogV;
  - conține? – 1 * E = E;
  - micșorează_val – logV * E = ElogV;
  - este_vidă? – 1 * V = V.

- Eficient dacă graful are arce puține comparativ cu numărul de noduri.

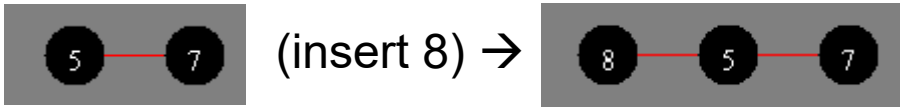


Heap Fibonacci

- Poate fi format din mai mulți arbori.
- Cheia unui părinte ≤ cheia oricărui copil.
- Fiind dat un nod u și un heap H:
  - p(u) – părintele lui u;
  - copil(u) – legătura către unul din copiii lui u;
  - st(u), dr(u) – legătura la frații din stânga și din dreapta (cei de pe primul nivel sunt legați între ei astfel);
  - grad(u) – numărul de copii ai lui u;
  - min(H) – cel mai mic nod din H;
  - n(H) – numărul de noduri din H.

Operatii Heap Fibonacci

- Inserare nod – O(1)
  - construiește un nou arbore cu un singur nod



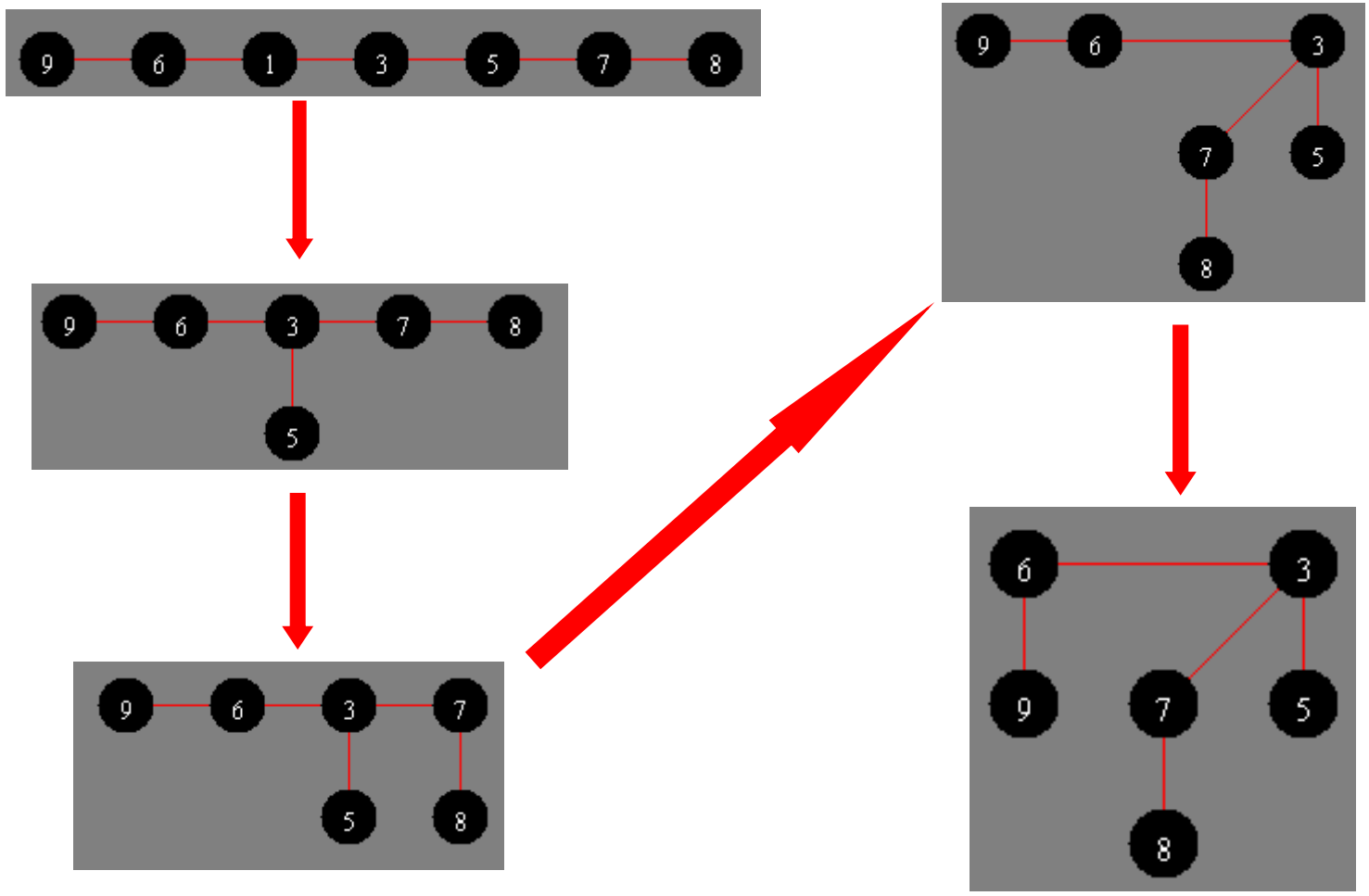
- Min – accesibil direct - min(H) – O(1)
- ExtrageMin O(logn) – cost amortizat!
  - Mută copiii minimului pe primul nivel;
  - **Consolidează** heap-ul.

Operatii Heap Fibonacci

- Consolidare Heap
  - Cât timp există 2 arbori cu grade egale Arb(x) și Arb(y), x < y:
    - Arb(y) transformat în copil al lui x;
    - grad[x] ++;
- Applet și implementare disponibile la [4].
- Gif + implementare puteți găsi la [5]



Consolidare Heap



Costuri Heap Fibonacci

- Costuri:
  - insert – 1 * V = V;
  - delete – logV * V = VlogV(amortizat!);
  - micșorează_val – 1 * E = E;
  - este_vidă? – 1 * V = V.
- Cea mai rapidă structură dpdv teoretic.

Concluzii Dijkstra (I)

- Dijkstra(G,s)
  - **Pentru fiecare nod u** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare nod v** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele

Complexitate?

Vectori - $O(V^2)$

HB - $O(E \log V)$

HF - $O(V \log V + E)$

Concluzii Dijkstra (II)

- Implementarea trebuie realizată în funcție de tipul grafului pe care lucrăm:
  - vectori pentru grafuri “dese”;
  - heap pentru grafuri “rare”.

- Heapul Fibonacci este mai eficient decât heapul binar dar mai dificil de implementat.



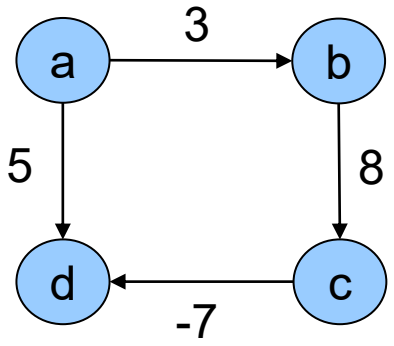
Concluzii Dijkstra (III)

- Implementarea trebuie realizată în funcție de tipul grafului pe care lucrăm:
  - vectori pentru grafuri “dese” - $O(V^2)$;
  - heap pentru grafuri “rare”: HB - O(E logV), HF - O(V log V+E)
- Heapul Fibonacci este mai eficient decât heapul binar dar mai dificil de implementat.



Problemă Dijkstra

- Exemplu rulare:
  - d[a] = 0; d[b] = d[c] = d[d] = ∞
  - d[b] = 3; d[d] = 5; d[c] = ∞
  - d[d] = 5; d[c] = 11;



  - **d este extras din coadă!** În momentul extragerii din coadă distanța pană la nodul d se consideră a fi calculată și a fi optimă.
  - Se extrage nodul c; d[d] nu va mai fi actualizată – nodul d fiind deja eliminat din coadă.
- → Algoritmul nu funcționează pentru grafuri ce conțin arce de cost negativ!

Exemplu practic – arce de cost negativ (I)

Currency conversion. Given currencies and exchange rates, what is best way to convert one ounce of gold to US dollars?

- 1 oz. gold ⇒ $327.25.
- 1 oz. gold ⇒ £208.10 ⇒ ⇒ $327.00. [208.10 × 1.5714]
- 1 oz. gold ⇒ 455.2 Francs ⇒ 304.39 Euros ⇒ $327.28. [455.2 × .6677 × 1.0752]

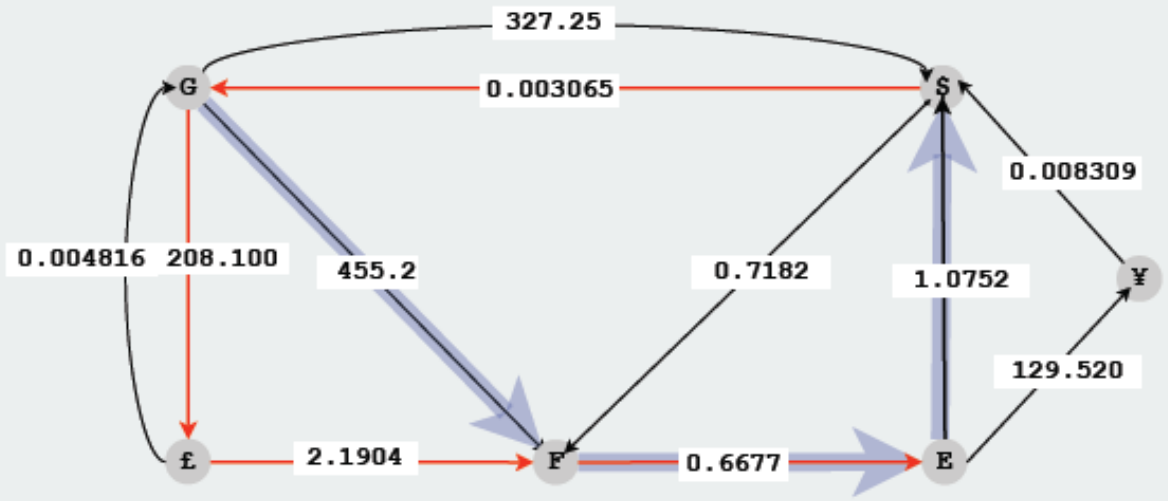
Currency	£	Euro	¥	Franc	$	Gold
UK Pound	1.0000	0.6853	0.005290	0.4569	0.6368	208.100
Euro	1.4599	1.0000	0.007721	0.6677	0.9303	304.028
Japanese Yen	189.050	129.520	1.0000	85.4694	120.400	39346.7
Swiss Franc	2.1904	1.4978	0.011574	1.0000	1.3929	455.200
US Dollar	1.5714	1.0752	0.008309	0.7182	1.0000	327.250
Gold (oz.)	0.004816	0.003295	0.0000255	0.002201	0.003065	1.0000

*slide din cursul de algoritmi de la Princeton – Sedgewick&Wayne[1]

Exemplu practic – arce de cost negativ (II)

Graph formulation.

- Vertex = currency.
- Edge = transaction, with weight equal to exchange rate.
- Find path that maximizes product of weights.



*slide din cursul de algoritmi de la Princeton – Sedgewick&Wayne[1]

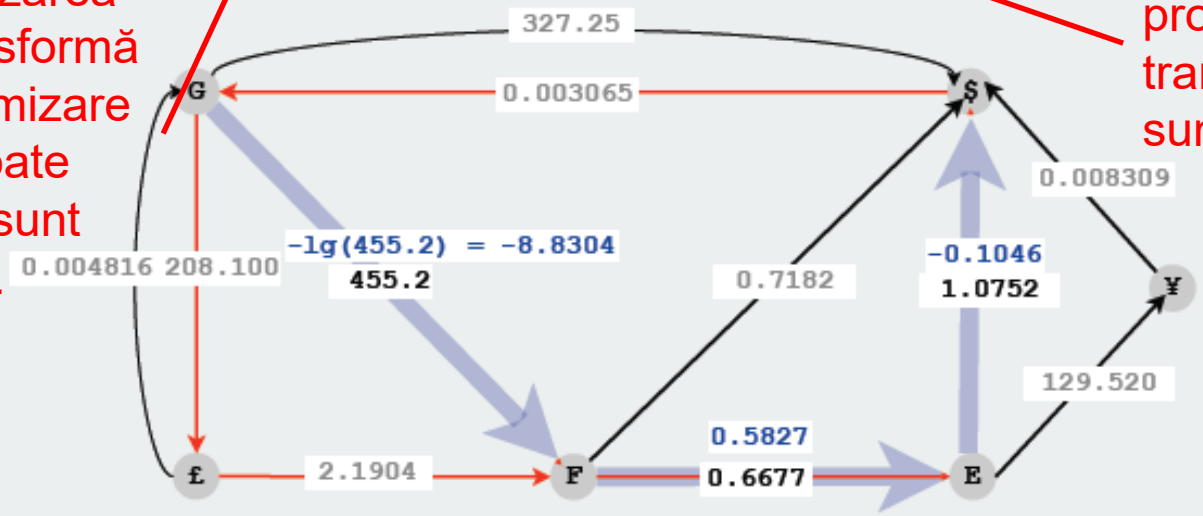
Exemplu practic – arce de cost negativ (III)

Reduce to shortest path problem by taking logs

- Let weight(v-w) = – lg (exchange rate from currency v to w)
- multiplication turns to addition
- Shortest path with costs c corresponds to best exchange sequence.

Maximizarea se transformă în minimizare dacă toate arcele sunt negate.

Prin logaritmare produsul se transformă în sumă.

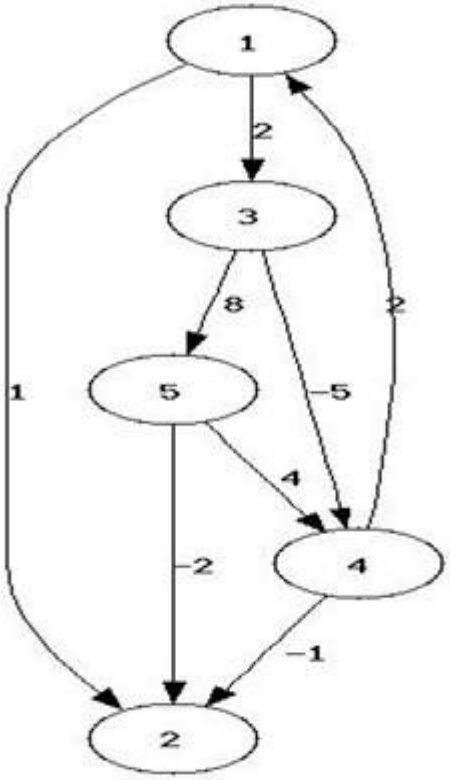


*slide din cursul de algoritmi de la Princeton – Sedgewick&Wayne[1]

Cicluri de cost negativ

$$\delta(u, v) = \begin{cases} \Sigma\, w(x,y),\ (x,y) \in u..v \\ (u..v \text{ fiind drumul optim}); \\ \\ \infty, \text{ dacă nu există drum} \\ u..v. \end{cases}$$

- Dacă există pe drumul u..v un ciclu de cost negativ x..y →
  - δ(u,v) = δ(u,v) + cost(x..y) < δ(u,v)
  - → valoarea lui δ(u,v) va scădea continuu → costul este -∞
  - → δ(u,v) = -∞

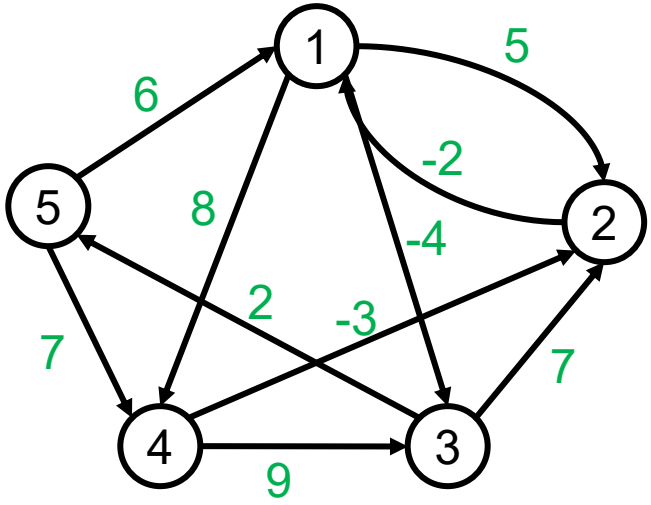


1-3-4 ciclu de cost negativ(-1)→ toate costurile din graf sunt -∞

Algoritmul Bellman-Ford

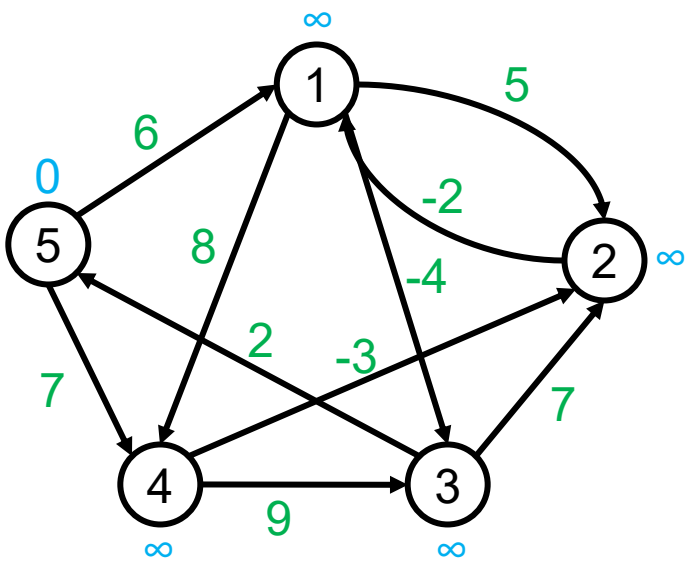
- BellmanFord(G,s) // G=(V,E),s=sursa
  - **Pentru fiecare** nod v (v ∈ V) // inițializări
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță de la s la s
  - **Pentru** *i* **de la** *1* **la** |*V*| *-1* // pentru fiecare pas pornind din s // spre restul nodurilor se încearcă construcția unor drumuri // optime de dimensiune i
    - **Pentru fiecare** (u,v) din E
      // pentru arcele ce pleacă de la nodurile deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

Exemplu Bellman-Ford (I)



- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

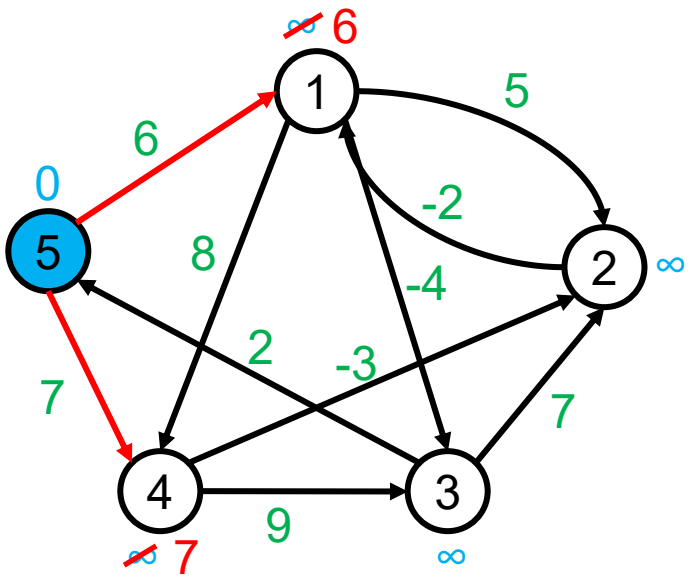
Exemplu Bellman-Ford (II)



d[1] = d[2] = d[3] = d[4] = d[5] = ∞
d[5] = 0

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

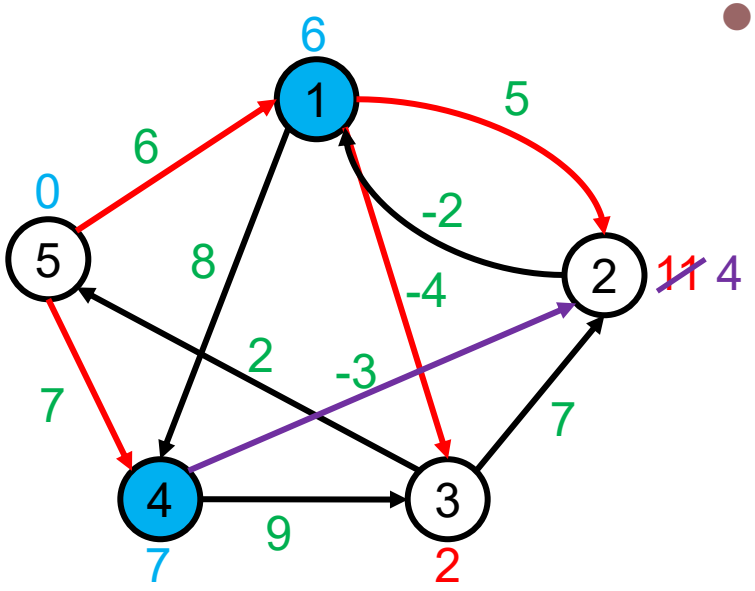
Exemplu Bellman-Ford (III)



Pas 1: relaxare (5,1) și (5,4)
d[1] = 6, p[1] = 5
d[4] = 7, p[4] = 5

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

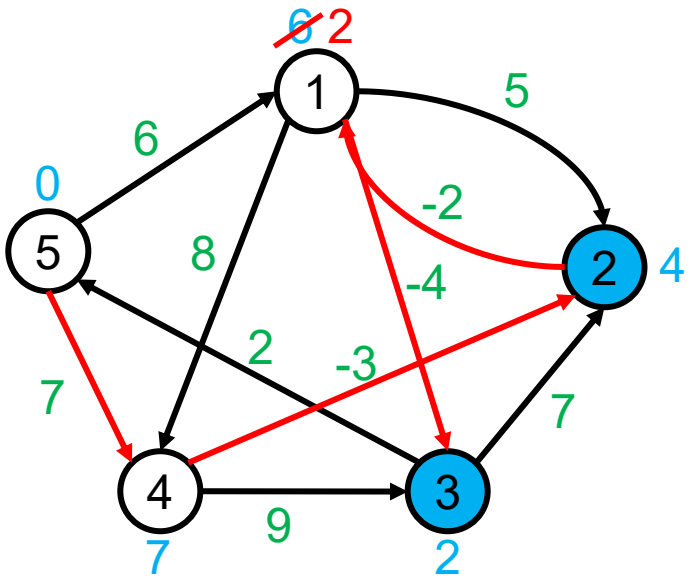
Exemplu Bellman-Ford (IV)



Pas 2: relaxare (1,2) și (1,3)
d[2] = 11, p[2] = 1
d[3] = 2, p[3] = 1
relaxare (4,2)
d[2] = 4, p[2] = 4

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

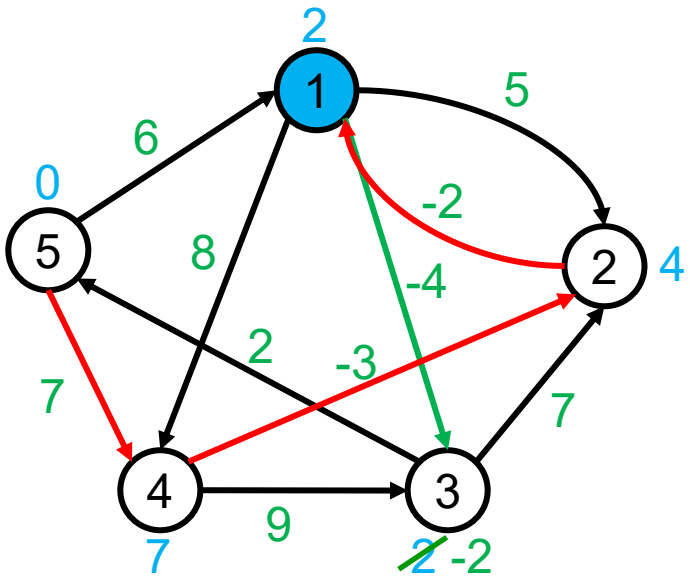
Exemplu Bellman-Ford (V)



Pas 3: relaxare (2,1)
d[1] = 2, p[1] = 2

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

Exemplu Bellman-Ford (VI)



Pas 4: relaxare (1,3)
d[3] = -2, p[3] = 1

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

Complexitate Bellman-Ford

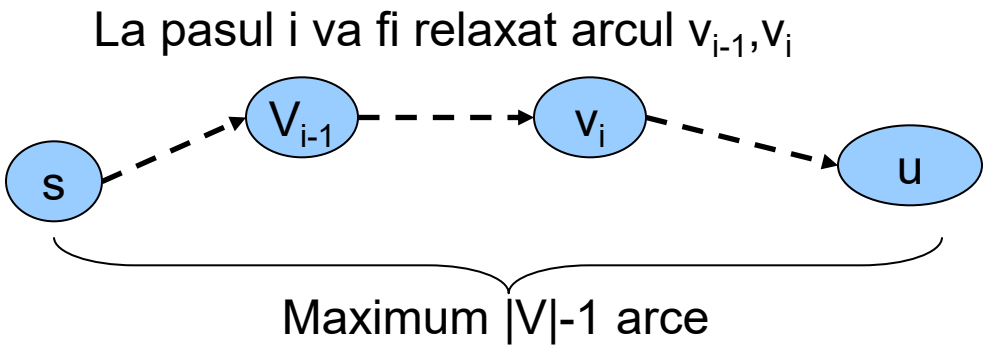
- cazul defavorabil:
  - **Pentru** i **de la** 1 **la** |V| - 1 —— V
    - **Pentru fiecare** (u,v) din E —— * E
      - **Dacă** d[v] > d[u] + w(u,v) **atunci**
        - d[v] = d[u] + w(u,v);
        - p[v] = u;

O(VE)

Corectitudine Bellman-Ford (I)

- Lemă 25.12: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată; dacă G nu conține ciclu de cost negativ atunci după |V| - 1 iterații ale relaxării fiecărui arc avem d[v] = δ(s,v) pentru ∀ v ∈ R(s).

- Dem prin inducție:
  - Fie s = $v_0, v_1 \ldots v_k$ = u un drum minim în graf cu k ≤ |V| - 1.



Corectitudine Bellman-Ford (II)

- Demonstrăm că în pasul i: $d[v_i] = \delta(s,v_i)$ și se menține până la sfârșit.
- P_0: (inițializare) → $d[s] = d[v_0] = 0 = \delta(s,s) = \delta(s,v_0)$ și conf. Lema 25.5, relația se menține până la sfârșit.
- $P_{i-1} \rightarrow P_i$:
  - P_{i-1}: $d[v_{i-1}] = \delta(s,v_{i-1})$,
  - În pasul i se relaxează arcul (v_{i-1},v_i), => conf. Lema 25.7 => $d[v_i] = d[v_{i-1}] + w(v_{i-1},v_i) = \delta(s,v_{i-1}) + w(v_{i-1},v_i) = \delta(s,v_i)$. Conf. Lema 25.5, relația se menține până la sfârșit.
  - Cum $i \in (1,|V|-1)$ → relația e adevărată pentru toate nodurile accesibile din s → $d[v] = \delta(s,v)$, $\forall\ v \in R(s)$.

Corectitudine Bellman-Ford (III)

- Teorema. G = (V,E), w : E → $\Re$ funcție de cost asociată. Algoritmul Bellman-Ford aplicat acestui graf plecând din sursa s nu returnează EROARE dacă G nu conține cicluri negative, iar la terminare d[v] = δ(s,v) pentru $\forall$ v $\in$ V. Dacă G conține cel puțin un ciclu negativ accesibil din s, atunci algoritmul întoarce EROARE.

- Dem: pe baza Lemei 25.12.
  - Dacă $\nexists$ ciclu negativ:
    - d[v] = δ(s,v) $\forall$ v $\in$ R(s)
    - d[v] = δ(s,v) = ∞, $\forall$ v $\notin$ R(s) (inițializare)
    - → d[v] ≤ d[u] + w(u,v) → nu se întoarce eroare (conf. Lema 25.3)
  - Dacă $\exists$ ciclu negativ → în cei |V| - 1 pași se scad costurile drumurilor, iar în final ciclul se menține → Eroare

Optimizări Bellman-Ford

- Observații!
  - 1) Dacă d[v] nu se modifică la pasul i atunci nu trebuie sa relaxăm niciunul din arcele care pleacă din v la pasul i + 1.
  - 2) => păstrăm o coadă cu vârfurile modificate (o singură copie).

Bellman-Ford optimizat 1

- BellmanFordOpt1(G,s)
  - **Pentru fiecare nod v** (v ∈ V)
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0;
  - **Pentru** i **de la** 1 **la** |V| -1
    - **Dacă nu** s-a modificat nicio distanță în iterația anterioară
      - **Break**
    - **Pentru fiecare nod u** (u ∈ V)
      - **Dacă** d[u] a fost modificat în iterația anterioară (i anterior)
        - **Pentru fiecare nod v** (v ∈ succesorilor lui u)
          - Dacă d[v] > d[u] + w(u,v) **atunci** // relaxez arcele ce pleacă din u
            - d[v] = d[u] + w(u,v);
            - p[v] = u;
- Ce se întâmplă cu cicluri negative?

Bellman-Ford optimizat 2

- BellmanFordOpt2(G,s)
  - **Pentru fiecare nod v** (v ∈ V)
    - d[v] = ∞;
    - p[v] = null;
    - marcat[v] = false; // marcăm nodurile pentru care am făcut relaxare
  - Q = ∅; // coadă cu priorități
  - d[s] = 0; marcat[s] = true; Introdu(Q,s);
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); marcat[u] = false; // extrag minimul
    - **Pentru fiecare nod v** (v ∈ succesorilor lui u)
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // relaxez arcele ce pleacă din u
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
        - **Dacă** (marcat[v] == false) {marcat[v] = true; Introdu(Q,v);}
- Observație: nu mai detectează cicluri negative!

ÎNTREBĂRI?



Bibliografie curs 9

[1] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*

[3] Giumale – Introducere în Analiza Algoritmilor cap. 5.4, 5.4.2